

TRAVAIL DE GENIE LOGICIEL



ISIG - GOMA

INSTITUT SUPERIEUR
D'INFORMATIQUE ET DE GESTION
BP : 841 GOMA

<http://www.isig.ac.cd>

E-mail: info@isig.ac.cd

REPUBLIQUE DEMOCRATIQUE DU CONGO
ENSEIGNEMENT SUPERIEUR ET UNIVERSITAIRE

E.S.U

Conception d'un système de Gestion des contraventions routières.

Effectué par :

1. DIMANDJA LUNYENGU Joseph
2. KIZYELE KABASELE METUSCHELAH
3. ZAGABE AGANZE Esther
4. MONGI JACQUES BRIGANCE

Enseignant responsable :

CT. ISAMUNA NKEMBO Josué

Promotion : LIC3-LIAGE ET LIC4-LSI

Option : SI

Année Académique

2025 - 2026

INTRODUCTION

Aujourd'hui, la modernisation des services publics passe inévitablement par le numérique. Dans le secteur de la sécurité routière, l'automatisation de la verbalisation ne représente plus seulement un confort technique, mais une réelle nécessité pour optimiser le travail des agents sur le terrain. C'est dans ce cadre que nous avons conçu et développé l'application « ***Gestion des contraventions routières*** », un projet réalisé dans le cadre du cours de Génie Logiciel pour les cycles L3-LIAGE et L4-LSI.

Traditionnellement, la saisie des procès-verbaux d'infraction routière repose sur des supports papier, entraînant des risques élevés d'erreurs matérielles, des pertes de documents, et des délais critiques dans le recouvrement des amendes. Notre solution propose une plateforme centralisée permettant de séparer les responsabilités entre deux acteurs majeurs : l'Agent verbalisateur sur le terrain et l'Administrateur au sein du bureau central. L'objectif de ce document est de fournir une documentation complète couvrant tout le cycle de vie du projet : de l'analyse des besoins à la validation en passant par l'explication détaillée de l'architecture logicielle retenue.

Chapitre I : ANALYSE FONCTIONNELLE ET MODELISATION UML

Avant de nous lancer dans le code, nous avons pris le temps de poser les bases du système. Cette phase d'analyse est essentielle en Génie logiciel pour traduire des besoins concrets en une structure informatique logique, formalisée dans notre travail par les diagrammes UML.

Pour que le système reste sécurisé et simple à utiliser, nous avons mis l'application en deux espaces bien distincts :

- **L'Agent Routier** : Équipé de son smartphone ou face à son ordinateur, il accède à une interface fluide pour enregistrer une infraction en direct. Il saisit les informations du véhicule et du conducteur, puis génère immédiatement un avis de contravention ;
- **L'Administrateur Système** : C'est le gestionnaire de la plateforme. Il possède une vue d'ensemble sur l'activité, gère les accès des agents, suit l'état des paiements, valide les quittances et veille au bon fonctionnement technique de l'application.

Sur le terrain, la rapidité est primordiale. Lorsqu'un agent intercepte un véhicule, il commence par saisir la plaque d'immatriculation. Notre système interroge alors instantanément la base de données. Si le véhicule ou le conducteur n'existent pas encore dans nos fichiers, l'application les enregistre et les synchronise automatiquement en arrière-plan. Cela permet à l'agent de continuer sa saisie sans subir le moindre blocage technique. A cela, nous avons élaboré différents diagrammes UML pour la modélisation de notre travail pour permettre une base de communication entre étude fonctionnelle et développement du projet informatique.

Le diagramme de cas d'utilisation représente la structure des grandes fonctionnalités nécessaires aux utilisateurs du système. C'est ici où s'assure la relation entre l'utilisateur et les objets que le système met en œuvre.

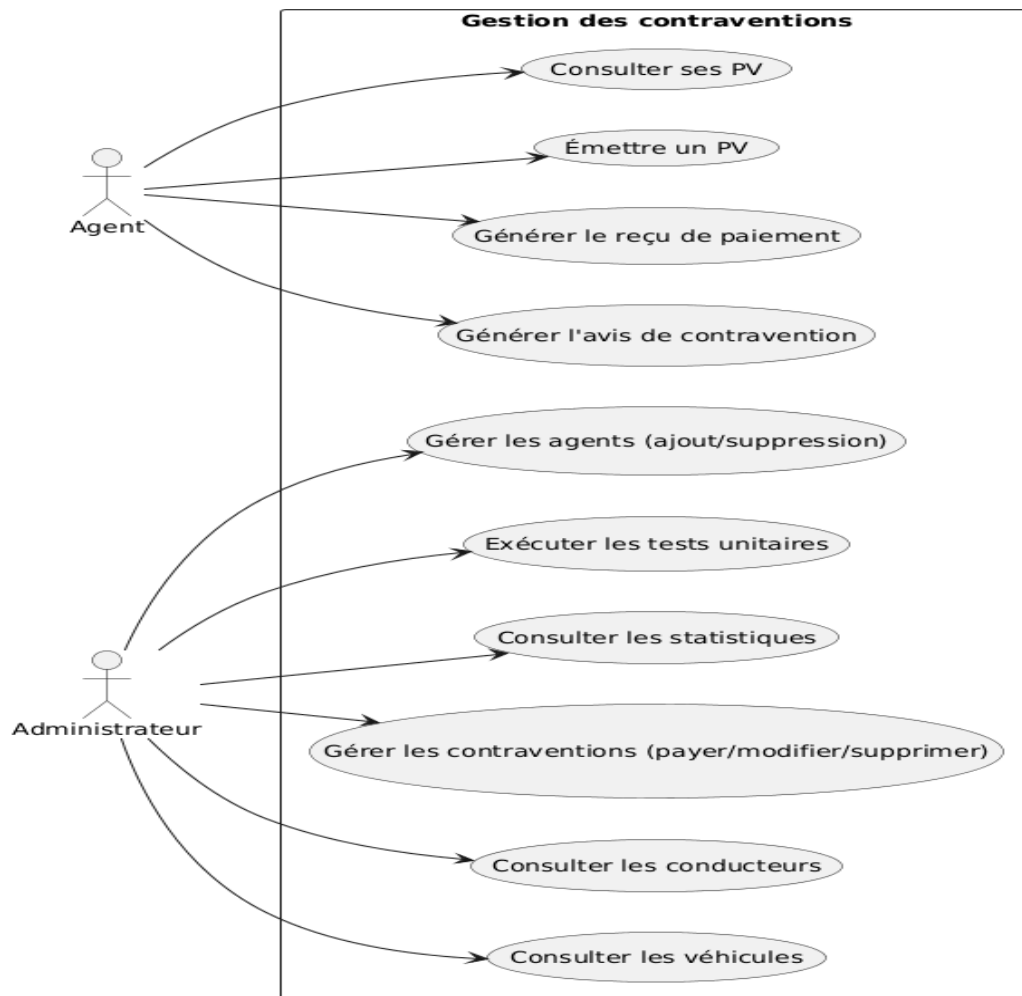


Figure 1. Diagramme des cas d'utilisation

Le diagramme de classes est généralement considéré comme le plus important dans un développement orienté objet. Il représente l'architecture conceptuelle du système : il décrit les classes que le système utilise, ainsi que leurs liens.

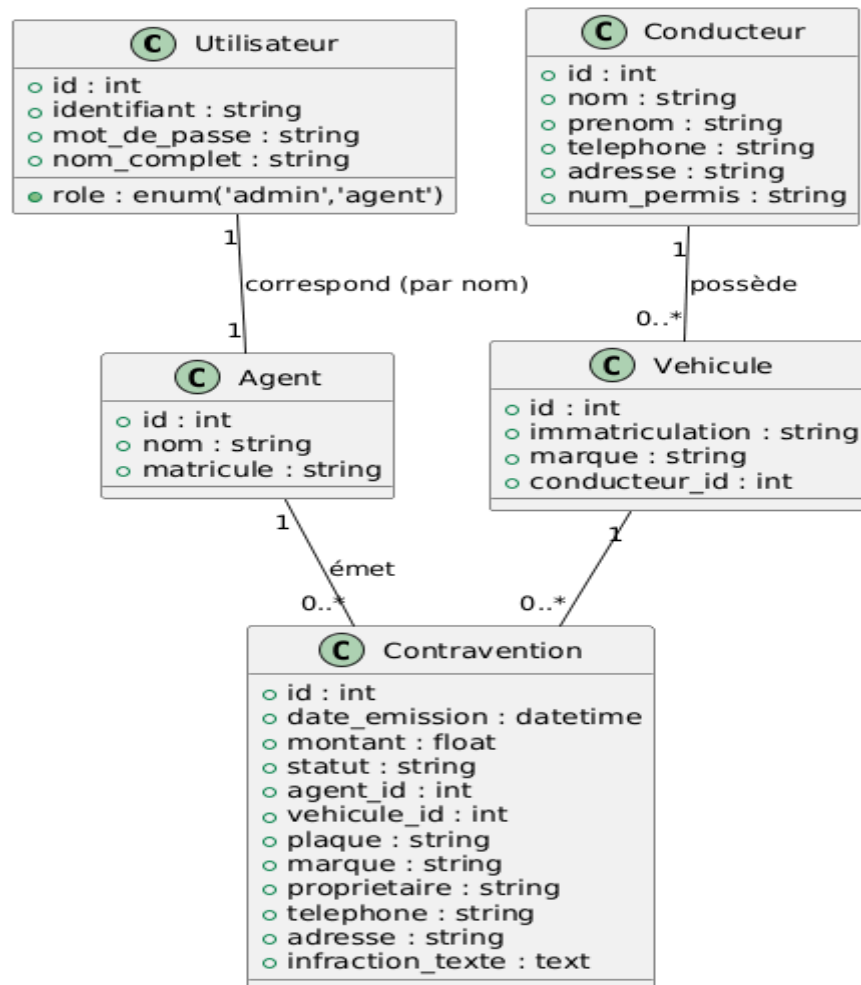


Figure 2. Diagramme de classes

Le diagramme d'activités n'est autre que la transcription dans UML de la représentation du processus telle qu'elle a été élaborée lors du travail qui a préparé la modélisation : il montre l'enchaînement des activités qui concourent au processus.

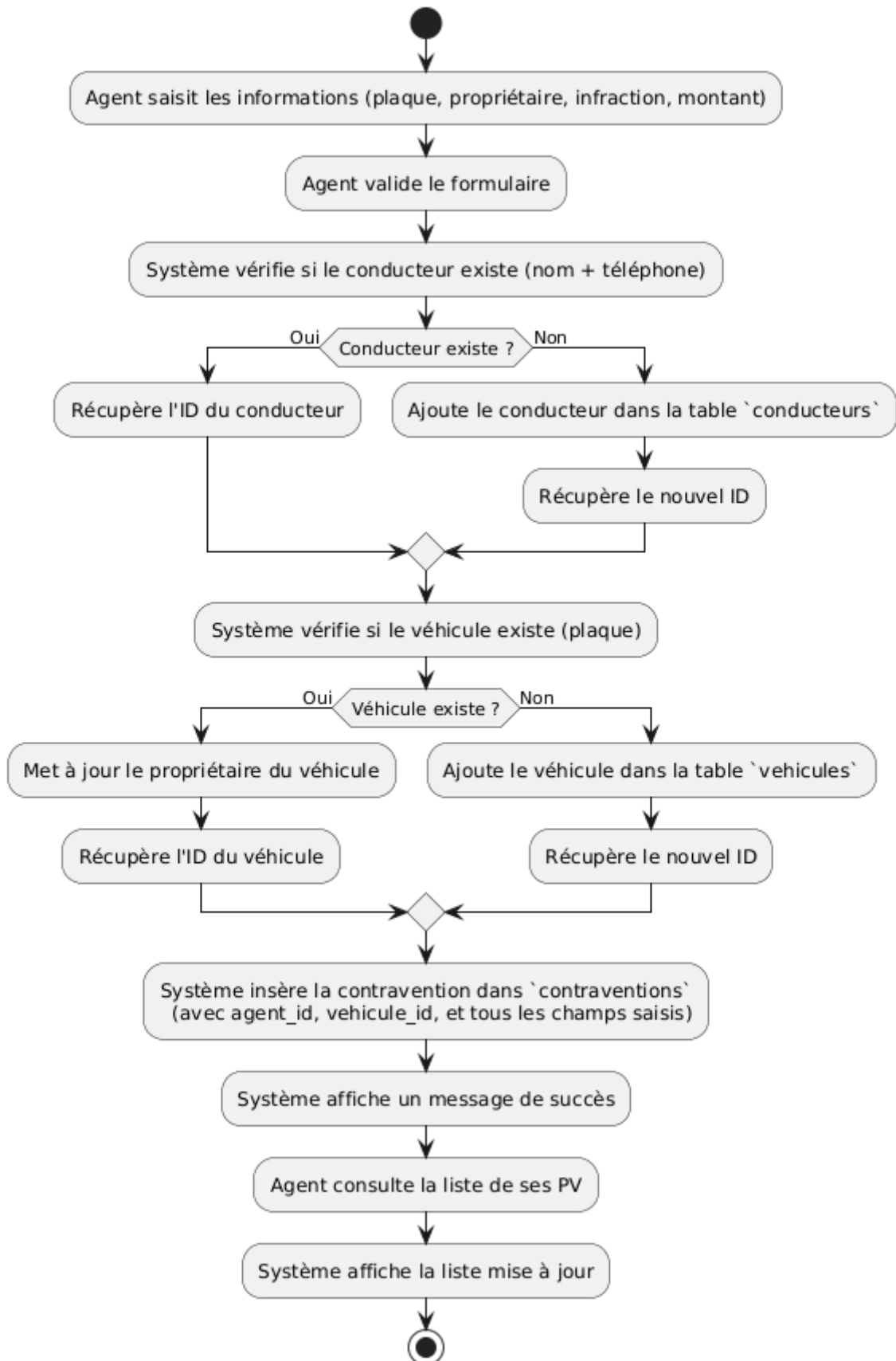


Figure 3. Diagramme d'activités

Le diagramme de séquence représente la succession chronologique des opérations réalisées par un acteur. Il indique les objets que l'acteur va manipuler et les opérations qui font passer d'un objet à l'autre.

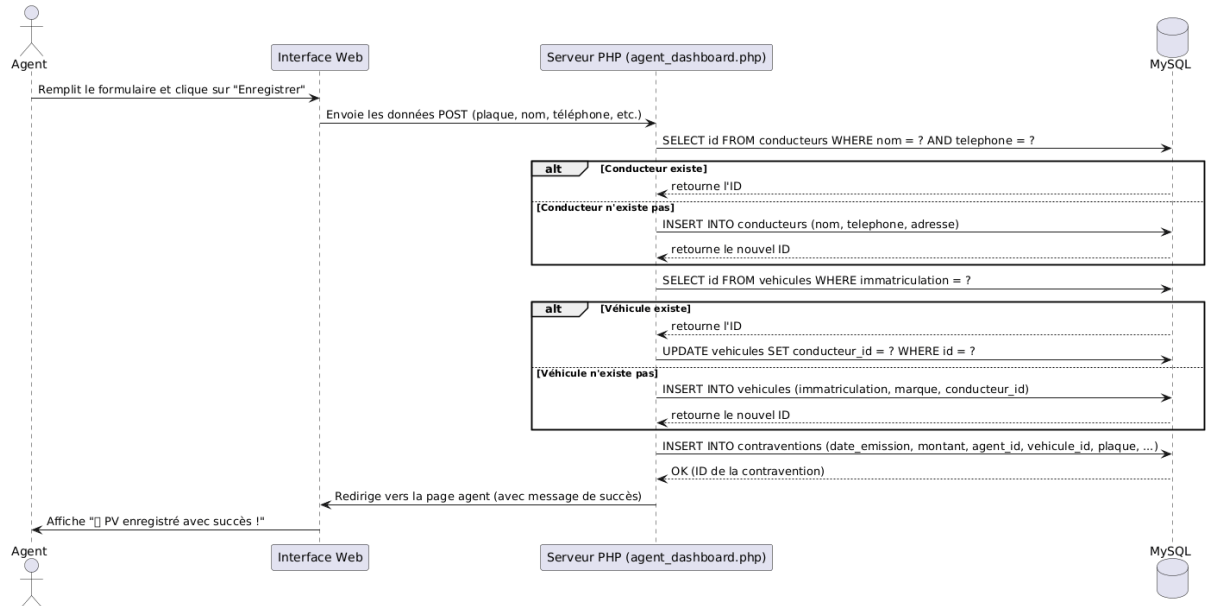


Figure 4. Diagramme de séquences

Le diagramme de déploiement décrit la disposition physique des ressources matérielles qui composent le système et montre la répartition des composants sur ces matériels. Chaque ressource étant matérialisée par un nœud, le diagramme de déploiement précise comment les composants sont répartis sur les nœuds et quelles sont les connexions entre les composants ou les nœuds.

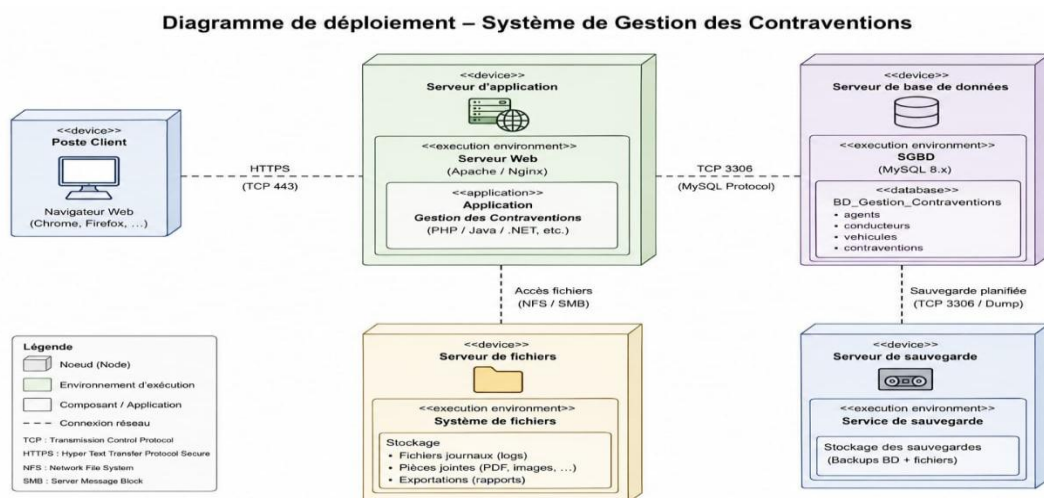


Figure 5. Diagramme de déploiement

Chapitre II : ARCHITECTURE LOGICIELLE ET STRUCTURE DE LA BASE DE DONNEES

Nous avons opté pour une architecture modulaire en **PHP** avec à une base de données relationnelle **MySQL** gérée via **XAMPP**. La base de données repose sur quatre tables interconnectées par des clés étrangères pour éviter toute perte de cohérence. Une attention particulière a été portée à la sécurité : les mots de passe des agents ne sont jamais stockés en clair, mais protégés dès leur création afin de respecter les bonnes pratiques de développement. Ces tables sont les suivantes :

- **utilisateurs** : id, identifiant, mot_de_passe, role, nom_complet
- **agents** : id, nom, matricule
- **conducteurs** : id, nom, prenom, telephone, adresse, num_permis
- **vehicules** : id, immatriculation, marque, conducteur_id
- **contraventions** : id, date_emission, montant, statut, vehicule_id, agent_id, plaque, marque, proprietaire, telephone, adresse, infraction_texte

Une bonne technique ne vaut rien sans une interface intuitive. Nous avons donc utilisé **HTML5** pour structurer nos pages et **CSS3** pour leur donner une apparence moderne et accessible.

Pour franchir l'espace de connexion, l'utilisateur doit s'authentifier. Le script vérifie les accès en base de données et initialise une session sécurisée en PHP. C'est cette session qui mémorise le rôle de l'utilisateur et lui interdit l'accès aux pages qui ne lui sont pas destinées.

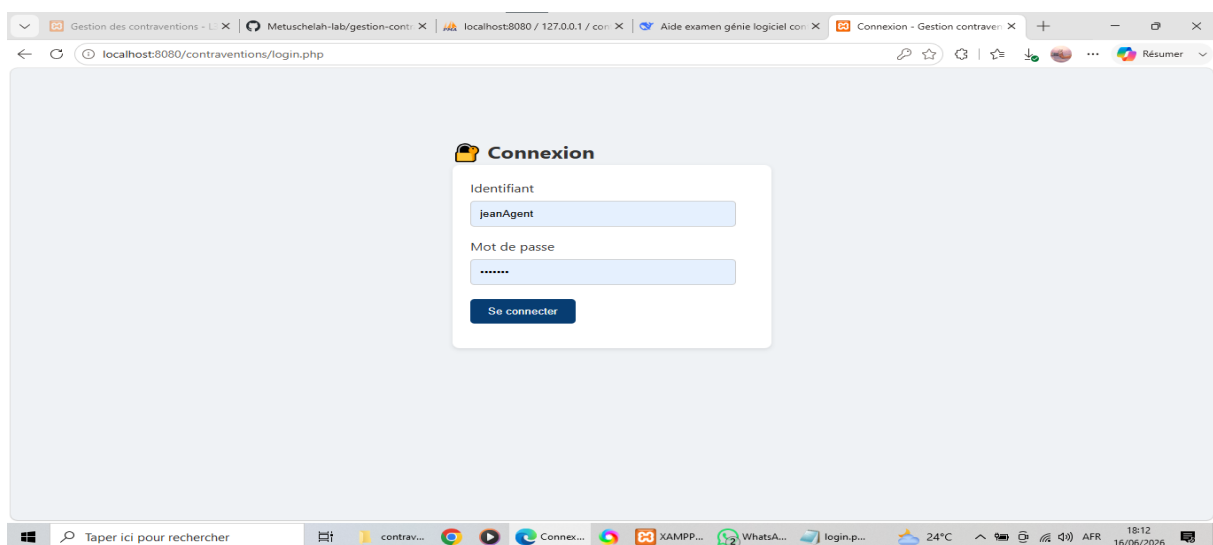


Figure 6. Page de connexion

L'interface de l'agent a été pensée pour être efficace. Le formulaire de verbalisation guide l'utilisateur pas à pas : saisie de la plaque, coordonnées du contrevenant, saisie du type d'infraction et du montant de l'amende.

Ravi de vous revoir, Officier Police Judiciaire ! Jean (Agent) - Matricule : 005 Déconnexion

✓ PV enregistré avec succès !

Dresser un Procès-Verbal

Plaque d'immatriculation (ex: 1234AB/22) Marque & Modèle (ex: Toyota Hilux) Nom du Propriétaire

Téléphone (ex: +243...) Adresse (Quartier, Avenue, N°)

Description de l'infraction (ex: Dépassement non autorisé)

Montant de l'amende (\$) Enregistrer et Émettre le PV

Registre des infractions enregistrées

ID	Date	Plaque	Infraction	Amende	Statut	Agent	Avis
#21	16/06/2026 18:09	123AA	Exces	120.00 \$	Émis	005	Avis

1 PV émis

Figure 7. Tableau de bord pour agent.

L'espace Administrateur constitue le centre de contrôle et de décision du système. Contrairement à l'interface de l'agent qui est orientée vers la saisie rapide sur le terrain, l'interface administrative offre une visibilité complète sur l'ensemble du parc automobile, des conducteurs identifiés et des performances des agents. Elle centralise les données pour piloter l'activité et valider les processus financiers.

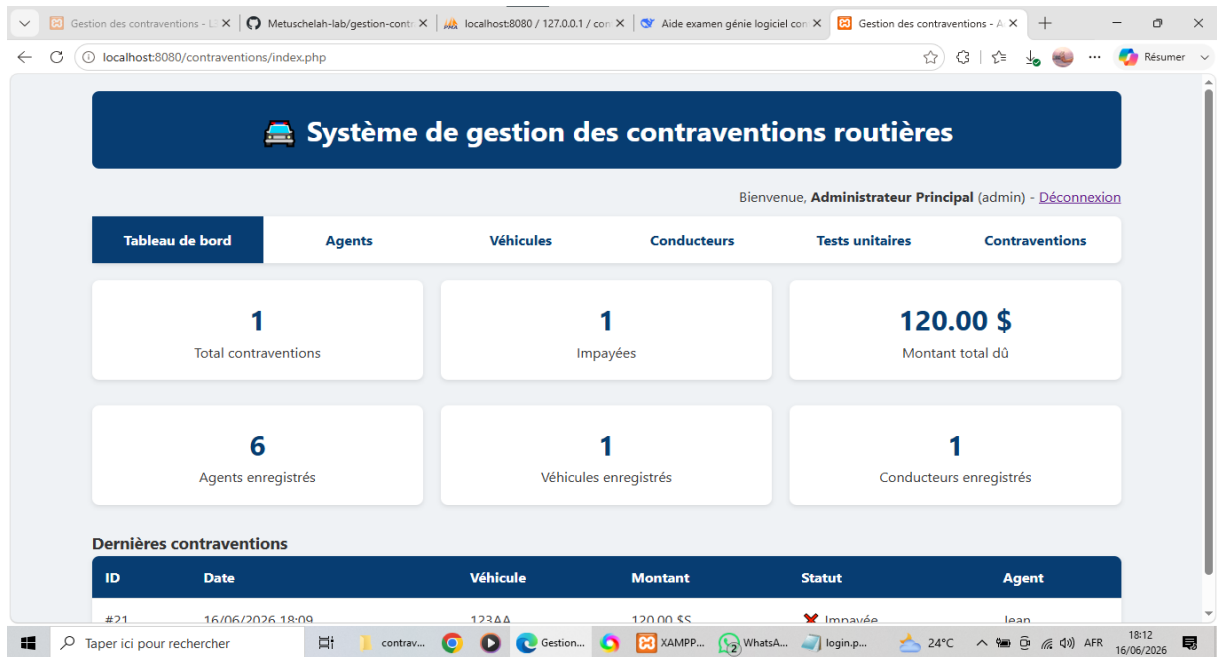


Figure 8. Tableau de bord pour admin

Le flux de travail se termine par l'édition de documents clairs. Le fichier « avis.php » génère la notification d'infraction propre et lisible à remettre au conducteur.



Figure 9. Avis de contravention imprimable

En parallèle, « **recu.php** » permet à l'administration d'éditer une quittance officielle dès qu'une amende est réglée.

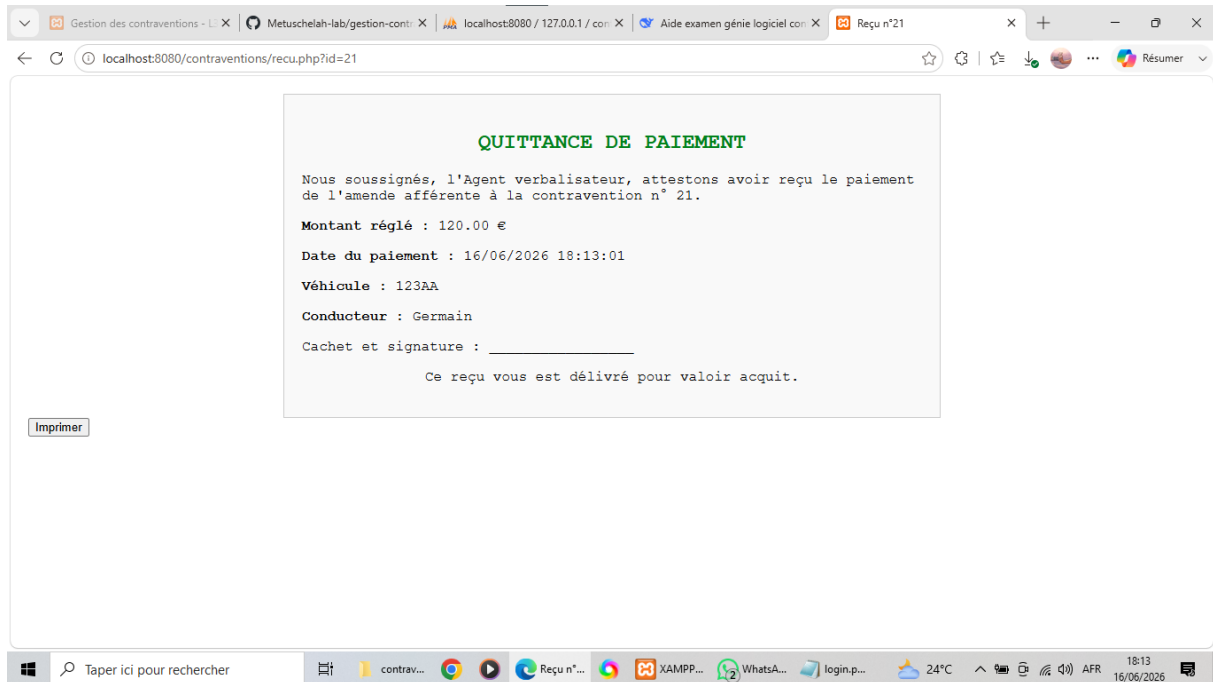


Figure 10. Quittance de paiement (Reçu)

Nous avons réalisé un seul test unitaire sur la création d'une contravention afin de s'assurer qu'une nouvelle contravention est correctement enregistrée dans la base de données avec les informations nécessaires.

```
CREATE TABLE `contraventions` (
  `date_emission` datetime DEFAULT NULL,
  `montant` decimal(10,2) DEFAULT NULL,
  `statut` varchar(20) DEFAULT 'impayee',
  `vehicule_id` int(11) DEFAULT NULL,
  `agent_id` int(11) DEFAULT NULL,
  `plaque` varchar(20) DEFAULT NULL,
  `marque` varchar(50) DEFAULT NULL,
  `proprietaire` varchar(100) DEFAULT NULL,
  `telephone` varchar(20) DEFAULT NULL,
  `adresse` varchar(255) DEFAULT NULL,
  `infraction_texte` text DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_general_ci;

--
-- Déchargement des données de la table `contraventions`
--

INSERT INTO `contraventions` (`id`, `date_emission`, `montant`, `statut`, `vehicule_id`, `agent_id`, `plaque`, `marque`, `proprietaire`
(21, '2026-06-16 18:09:53', 120.00, 'payee', 5, 17, '123AA', 'AudIRS3', 'Germain', '+24309987633', 'Goma ville', 'Exces');
```

Figure 11. Test unitaire pour la création d'une contravention

Conclusion

Mener à bien ce projet de gestion des contraventions routières a été une opportunité concrète d'appliquer les concepts théoriques de notre cours de Génie Logiciel. En associant une modélisation UML claire à un développement modulaire en PHP 8.2 et MySQL, nous avons pu concevoir un outil à la fois sécurisé, logique et fonctionnel. Les objectifs que nous nous étions fixés sont atteints. La séparation des rôles est parfaitement respectée, l'enregistrement des données est fluide pour l'agent, et la fiabilité de nos fonctions majeures est validée par des tests automatiques intégrés.